

2 Machines & Models

Efficient Algorithms · Wintersemester 2026–27

Prof. Dr. Sebastian Wild

CS566 (Wintersemester 2026–27)
Philipps-Universität Marburg
version 2026-10-13 14:07

Learning Outcomes

Unit 2: ***Machines & Models***

1. Understand the difference between empirical *running time* and algorithm *analysis*.
2. Understand *worst/best/average case* models for input data.
3. Know the *RAM machine* model.
4. Know the definitions of *asymptotic notation* (Big-Oh classes and relatives).
5. Understand the reasons to make *asymptotic approximations*.
6. Be able to *analyze* simple *algorithms*.

Outline

2 Machines & Models

2.1 Algorithm analysis

2.2 The RAM Model

2.3 Asymptotics & Big-Oh

2.4 Asymptotic Pitfalls

2.5 Teaser: The Maximum subarray problem

2.6 Teaser: Algorithm Design in Action

What is an algorithm?

An algorithm is a sequence of instructions.

think: recipe



More precisely:

e. g. Python script

1. mechanically executable
   ⇝ no "common sense" needed
2. finite description ≠ finite computation!
3. solves a *problem*, i. e., a class of problem instances

$x + y$, not only $17 + 4$

- input-processing-output abstraction



Typical example: *bubblesort*

⇝ not a specific program
but the underlying idea

What is a data structure?

A data structure is

1. a rule for **encoding data** (in computer memory), plus
2. **algorithms** to work with it (queries, updates, etc.)

typical example: *binary search tree*



2.1 Algorithm analysis

Good algorithms

Our goal: Find good (best?) algorithms and data structures for a task.

Good "usually" means

- fast running *time* (can be complicated in distributed systems)
- moderate memory *space* usage

Algorithm analysis is a way to

- compare different algorithms,
- predict their performance in an application

Running time experiments

Why not simply run and time it?



- results only apply to
  - single *test* machine
  - tested inputs
  - tested implementation
  - ...
  - $\neq$ *universal truths*

- instead: consider and analyze algorithms on an abstract machine
  - $\rightsquigarrow$ provable statements for model
  - $\rightsquigarrow$ testable model hypotheses

survives Pentium 4

$\rightsquigarrow$ Need precise model of machine (costs), input data and algorithms.

Data Models

Algorithm analysis typically uses one of the following simple data models:

- **worst-case performance:**
  consider the *worst* of all inputs as our cost metric
- **best-case performance:**
  consider the *best* of all inputs as our cost metric
- **average-case performance:**
  consider the average/expectation of a *random* input as our cost metric

Usually, we apply the above for *inputs of same size* n.

$\rightsquigarrow$ performance is only a **function of** n.

2.2 The RAM Model

Clicker Question



What is the cost of *adding* two d-digit integers?
(For example, for $d = 5$, what is $45\,235 + 91\,342$?)

- **A** constant time
- **B** logarithmic in d
- **C** proportional to d
- **D** quadratic in d
- **E** no idea what you are talking about



→ *sli.do/effalg*

Clicker Question



What is the cost of *adding* two d-digit integers?
(For example, for $d = 5$, what is $45\,235 + 91\,342$?)

- **A** constant time ✓
- **B** ~~logarithmic in d~~
- **C** proportional to d ✓
- **D** ~~quadratic in d~~
- **E** no idea what you are talking about ✓



→ *sli.do/effalg*

Machine models

The machine model decides

- what algorithms are possible
- how they are described (= programming language)
- what an execution *costs*

Goal: Machine models should be
detailed and powerful enough to reflect actual machines,
abstract enough to unify architectures,
simple enough to analyze.

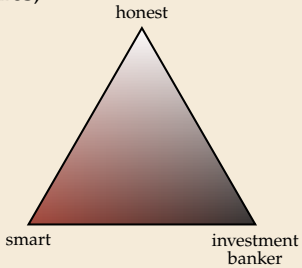
Machine models

The machine model decides

- what algorithms are possible
- how they are described (= programming language)
- what an execution *costs*

Goal: Machine models should be
detailed and powerful enough to reflect actual machines,
abstract enough to unify architectures,
simple enough to analyze.

⇝ usually some compromise is needed

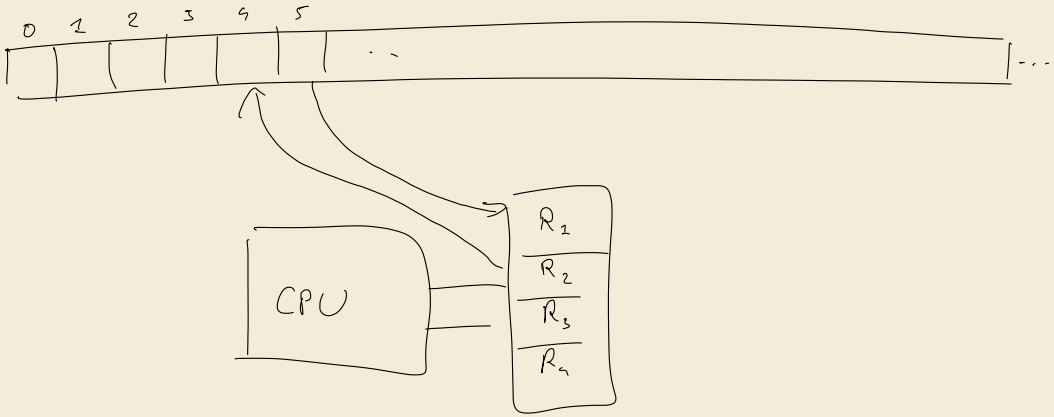


Random Access Machines

Random access machine (RAM)

more detail in §2.2 of *Sequential and Parallel Algorithms and Data Structures* by Sanders, Mehlhorn, Dietzfelbinger, Dementiev

- unlimited *memory* `MEM[0]`, `MEM[1]`, `MEM[2]`, ...
- fixed number of *registers* $R_1, \ldots, R_r$ (say $r = 100$)



Random Access Machines

Random access machine (RAM)

more detail in §2.2 of *Sequential and Parallel Algorithms and Data Structures* by Sanders, Mehlhorn, Dietzfelbinger, Dementiev

- unlimited *memory* $\texttt{MEM}[0], \texttt{MEM}[1], \texttt{MEM}[2], \ldots$
- fixed number of *registers* $R_1, \ldots, R_r$ (say $r = 100$)
- memory cells $\texttt{MEM}[i]$ and registers R_i store w-bit integers, i. e., numbers in $[0..2^w - 1]$
  w is the word width/size; typically $\boxed{w \propto \lg n} \rightsquigarrow 2^w \approx n$

Random Access Machines

Random access machine (RAM)

more detail in §2.2 of *Sequential and Parallel Algorithms and Data Structures* by Sanders, Mehlhorn, Dietzfelbinger, Dementiev

- unlimited *memory* $\texttt{MEM}[0], \texttt{MEM}[1], \texttt{MEM}[2], \ldots$
- fixed number of *registers* $R_1, \ldots, R_r$ (say $r = 100$)
- memory cells $\texttt{MEM}[i]$ and registers R_i store w-bit integers, i. e., numbers in $[0..2^w - 1]$
  w is the word width/size; typically $\boxed{w \propto \lg n}$ $\rightsquigarrow$ $2^w \approx n$
- Instructions:
  - load & store: $R_i := \texttt{MEM}[R_j]$ $\quad \texttt{MEM}[R_j] := R_i$
  - operations on registers: $R_k := R_i + R_j$ (arithmetic is *modulo* 2^w!)
    also $R_i - R_j$, $R_i \cdot R_j$, $R_i \operatorname{div} R_j$, $R_i \bmod R_j$
    C-style operations (bitwise and/or/xor, left/right shift)
  - conditional and unconditional jumps
- cost: number of executed instructions

we will see further models later

$\rightsquigarrow$ The RAM is the standard model for sequential computation.

RAM-Program Example

Example RAM program

```
1  // Assume: R_1 stores number N
2  // Assume: MEM[0..N) contains list of N numbers
3  R_2 := R_1;
4  R_3 := R_1 − 1;
5  R_5 := R_3 − 1;
6  R_4 := MEM[R_5];
7  R_6 := MEM[R_3];
8  if (R_4 ≤ R_6) goto line 11;
9  MEM[R_5] := R_6;
10 MEM[R_3] := R_4;
11 R_3 := R_3 − 1;
12 if (R_3 > 0) goto line 5;
13 R_2 := R_2 − 1;
14 if (R_2 > 0) goto line 4;
15 // Done:
```

Clicker Question



What algorithm does the RAM program on the previous slide implement?



→ *sli.do/effalg*

RAM-Program Example

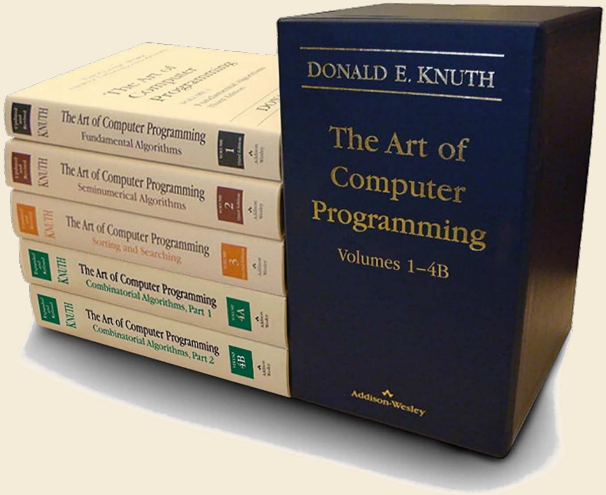
Example RAM program

```
1  // Assume: R_1 stores number N
2  // Assume: MEM[0..N) contains list of N numbers
3  R_2 := R_1;
4  R_3 := R_1 − 1;
5  R_5 := R_3 − 1;
6  R_4 := MEM[R_5];
7  R_6 := MEM[R_3];
8  if (R_4 ≤ R_6) goto line 11;
9  MEM[R_5] := R_6;
10 MEM[R_3] := R_4;
11 R_3 := R_3 − 1;
12 if (R_3 > 0) goto line 5;
13 R_2 := R_2 − 1;
14 if (R_2 > 0) goto line 4;
15 // Done: MEM[0..N) sorted
```

RAM-Program Example

Example RAM program

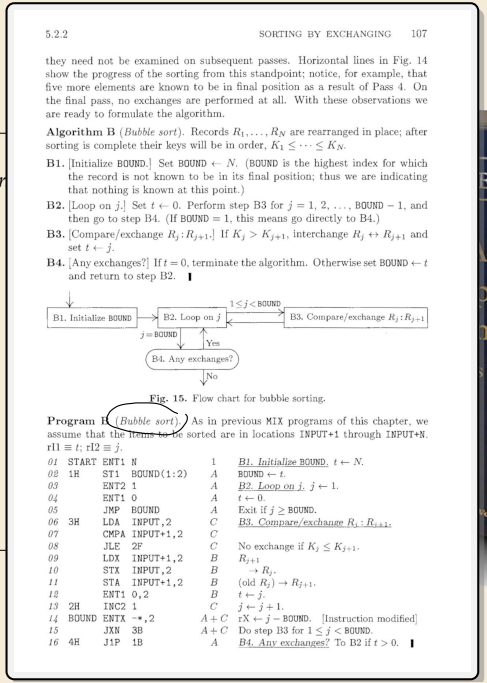
```
1  // Assume: R_1 stores number N
2  // Assume: MEM[0..N) contains list of N numbers
3  R_2 := R_1;
4  R_3 := R_1 − 1;
5  R_5 := R_3 − 1;
6  R_4 := MEM[R_5];
7  R_6 := MEM[R_3];
8  if (R_4 ≤ R_6) goto line 11;
9  MEM[R_5] := R_6;
10 MEM[R_3] := R_4;
11 R_3 := R_3 − 1;
12 if (R_3 > 0) goto line 5;
13 R_2 := R_2 − 1;
14 if (R_2 > 0) goto line 4;
15 // Done: MEM[0..N) sorted
```



RAM-Program Example

Example RAM program

1 // *Assume:* R_1 *stores number* N
2 // *Assume:* MEM[0..N] *contains list of* N *numbers*
3 $R_2 := R_1$;
4 $R_3 := R_1 - 1$;
5 $R_5 := R_3 - 1$;
6 $R_4 := \mathtt{MEM}[R_5]$;
7 $R_6 := \mathtt{MEM}[R_3]$;
8 **if** $(R_4 \leq R_6)$ goto line 11;
9 $\mathtt{MEM}[R_5] := R_6$;
10 $\mathtt{MEM}[R_3] := R_4$;
11 $R_3 := R_3 - 1$;
12 **if** $(R_3 > 0)$ goto line 5;
13 $R_2 := R_2 - 1$;
14 **if** $(R_2 > 0)$ goto line 4;
15 // *Done:* MEM[0..N] *sorted*



5.2.2 SORTING BY EXCHANGING 107

they need not be examined on subsequent passes. Horizontal lines in Fig. 14 show the progress of the sorting from this standpoint; notice, for example, that five more elements are known to be in final position as a result of Pass 4. On the final pass, no exchanges are performed at all. With these observations we are ready to formulate the algorithm.

Algorithm B (*Bubble sort*). Records $R_1, \ldots, R_N$ are rearranged in place; after sorting is complete their keys will be in order, $K_1 \leq \cdots \leq K_N$.

B1. [Initialize BOUND.] Set BOUND $\leftarrow N$. (BOUND is the highest index for which the record is not known to be in its final position; thus we are indicating that nothing is known at this point.)

B2. [Loop on j.] Set $t \leftarrow 0$. Perform step B3 for $j = 1, 2, \ldots,$ BOUND $- 1$, and then go to step B4. (If BOUND $= 1$, this means go directly to B4.)

B3. [Compare/exchange $R_j : R_{j+1}$.] If $K_j > K_{j+1}$, interchange $R_j \leftrightarrow R_{j+1}$ and set $t \leftarrow j$.

B4. [Any exchanges?] If $t = 0$, terminate the algorithm. Otherwise set BOUND $\leftarrow t$ and return to step B2. ∎

Fig. 15. Flow chart for bubble sorting.

Program B (*Bubble sort*). As in previous MIX programs of this chapter, we assume that the items to be sorted are in locations INPUT+1 through INPUT+N. rI1 $\equiv t$; rI2 $\equiv j$.

```
01  START ENT1 N              1     B1. Initialize BOUND. t ← N.
02  1H    ST1  BOUND(1:2)     A     BOUND ← t.
03        ENT2 1              A     B2. Loop on j. j ← 1.
04        ENT1 0              A     t ← 0.
05        JMP  BOUND          A     Exit if j ≥ BOUND.
06  3H    LDA  INPUT,2        C     B3. Compare/exchange R_j : R_{j+1}:
07        CMPA INPUT+1,2      C
08        JLE  2F             C     No exchange if K_j ≤ K_{j+1}.
09        LDX  INPUT+1,2      B     R_{j+1}
10        STX  INPUT,2        B       → R_j.
11        STA  INPUT+1,2      B     (old R_j) → R_{j+1}.
12        ENT1 0,2            B     t ← j.
13  2H    INC2 1              C     j ← j + 1.
14  BOUND ENTX *+*,2          A+C   rX ← j − BOUND.  [Instruction modified]
15        JXN  3B             A+C   Do step B3 for 1 ≤ j < BOUND.
16  4H    J1P  1B             A     B4. Any exchanges? To B2 if t > 0. ∎
```

Pseudocode

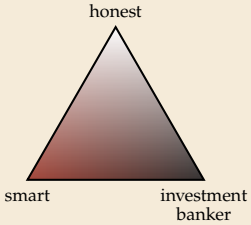
- Programs for the random-access machine are very low level and detailed

≈ assembly/machine language

Typical simplifications when describing and analyzing algorithms:

- more abstract *pseudocode* ← code that humans understand (easily)
  - control flow using **if**, **for**, **while**, etc.
  - variable names instead of fixed registers and memory cells
  - memory management (more below)
- count dominant *elementary operations* (e. g. memory accesses) instead of all RAM instructions

In both cases: We *can* go to full detail where needed/desired.



Pseudocode – Example

RAM-Program

```
1  // Bubblesort
2  // Assume: R_1 stores number N
3  // Assume: MEM[0..N) contains list of N numbers
4  R_2 := R_1;
5  R_3 := R_1 − 2;
6  R_4 := MEM[R_3];
7  R_5 := R_3 + 1;
8  R_6 := MEM[R_5];
9  if (R_4 ≤ R_6) goto line 12;
10 MEM[R_3] := R_6;
11 MEM[R_5] := R_4;
12 R_3 := R_3 − 1;
13 if (R_3 ≥ 0) goto line 6;
14 R_2 := R_2 − 1;
15 if (R_2 > 0) goto line 5;
16 // Done: MEM[0..N) sorted
```

Pseudocode Algorithm

```
1 procedure bubblesort(A[0..N)):
2     for i := N, N − 1, . . . , 1
3         for j := N − 2, N − 3, . . . , 0
4             if A[j] > A[j + 1]:
5                 Swap A[j] and A[j + 1]
6             end if
7         end for
8     end for
```

- ⇝ much more **readable**
- closer to modern high-level programming languages
- **but:** only allow primitive operations that correspond to $O(1)$ RAM instructions
  - ⇝ analysis

Memory management & Pointers

- A random-access machine is a bit like a bare CPU . . . without any operating system
  - ⇝ cumbersome to use
- All high-level programming languages / operating systems add *memory management*:

Memory management & Pointers

- A random-access machine is a bit like a bare CPU . . . without any operating system
  - ⇝ cumbersome to use
- All high-level programming languages / operating systems add *memory management*:
  - Instruction to *allocate* a contiguous piece of memory of a given size (like `malloc`).
    - used to allocate a `new` array (of a fixed size) or
    - a `new` object/record (with a known list of instance variables)
    - There's a similar instruction to `free` allocated memory again or an automated garbage collector.
  - ⇝ A *pointer* is a memory address (i. e., the i of $\texttt{MEM}[i]$).

Memory management & Pointers

- A random-access machine is a bit like a bare CPU . . . without any operating system
  - ⇝ cumbersome to use
- All high-level programming languages / operating systems add *memory management*:
  - Instruction to *allocate* a contiguous piece of memory of a given size (like `malloc`).
    - used to allocate a `new` array (of a fixed size) or
    - a `new` object/record (with a known list of instance variables)
    - There's a similar instruction to `free` allocated memory again or an automated garbage collector.
  - ⇝ A *pointer* is a memory address (i. e., the i of $\texttt{MEM}[i]$).
  - Support for procedures (a. k. a. functions, methods) calls including recursive calls
    - (this internally requires maintaining call stack)

Memory management & Pointers

- A random-access machine is a bit like a bare CPU . . . without any operating system
  - ⇝ cumbersome to use
- All high-level programming languages / operating systems add *memory management*:
  - Instruction to *allocate* a contiguous piece of memory of a given size (like `malloc`).
    - used to allocate a `new` array (of a fixed size) or
    - a `new` object/record (with a known list of instance variables)
    - There's a similar instruction to `free` allocated memory again or an automated garbage collector.
  - ⇝ A *pointer* is a memory address (i. e., the i of $\texttt{MEM}[i]$).
  - Support for procedures (a. k. a. functions, methods) calls including recursive calls
    - (this internally requires maintaining call stack)



We will mostly ignore *how* all this works here.